Forge Recursive Improvement Loop — Core Specification

Status: Draft v0.1 — 2026-03-08 **Type:** System extension (operates across all Forge components) **Depends on:** Azul (verification + XP + training pairs), ForgeAtlas (contract catalog), CONCORD (gate policies) **Strategic outcome:** Each cycle's output improves the next cycle's input, building a proprietary, codebase-specific intelligence asset

---

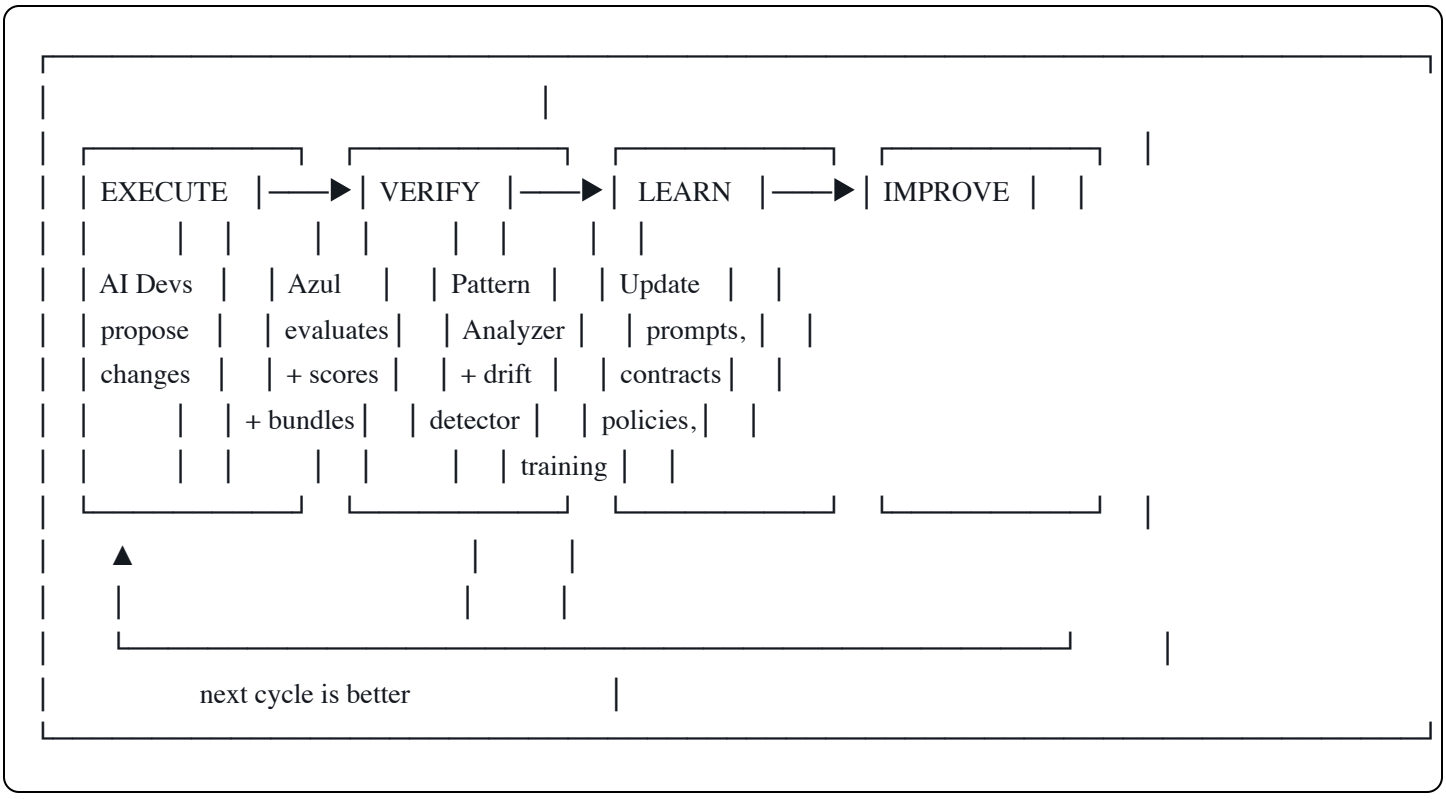
1 — Purpose

The Recursive Improvement Loop transforms the Forge ecosystem from a static verification system into a self-improving one. Every verification cycle produces structured evidence. That evidence is analyzed for patterns. Those patterns drive updates to prompts, contracts, policies, and training data. The updated system produces better work in the next cycle. Repeat.

The loop answers: "How do we ensure the team gets smarter every week, not just busier?"

---

2 — The Four Stages



EXECUTE — AI Devs produce work. Each change becomes an Azul ticket.

VERIFY — Azul runs every change through the full Forge stack. ReviewBundles, scores, ForgeGate decisions, XP awards, and training pairs are emitted as structured evidence.

LEARN — The Pattern Analyzer reads the accumulated evidence, computes quality metrics, detects drift, identifies recurring failure modes, and produces an improvement report.

IMPROVE — Findings from LEARN drive updates at four levels: prompts, contracts, policies, and training data. The updated system feeds back into EXECUTE.

---

3 — Stage 3: LEARN (Pattern Analyzer)

3.1 Metrics computed

The Pattern Analyzer reads Azul's ticket store, XP ledger, and ReviewBundles to compute:

Metric	Definition	Signal
Lead Agreement Rate	Percentage of AI-generated artifacts approved by human without modification	Top-line quality metric. Rising = system improving. Flat = loop not working.
Reject rate (by ticket_type)	REJECTED tickets / total tickets per type	Which work types the Devs struggle with most
Deny rate (by ForgeGate phase)	ForgeGate DENY decisions / total decisions per phase	Which governance rules Devs don't understand
Oracle mismatch frequency	Recurring oracle mismatches by action or resource	Behavioral expectations that need clearer specification
Score distribution	Histogram of total_score across tickets	Overall quality trend — is it shifting right over time?
Score trend	Moving average of total_score over time windows	Detecting gradual improvement or degradation
Failure mode clustering	Grouping FAILED tickets by error code and stage	Operational friction that needs infrastructure fixes
XP velocity	XP awarded per time window	Throughput of verified work
Distillation yield	Training pairs emitted / total tickets	Efficiency of the distillation pipeline
Dev performance ranking	Average score by agent_class or session pattern	Identifying which Dev configurations produce the best work

3.2 Lead Agreement Rate — detailed design

This is the single most important metric in the loop. It measures whether the system's output matches human expectations without correction.

What counts as an agreement:

- Human approves an ActionContract stub without editing any field → agreement
- Human approves a ReviewBundle verdict without overriding → agreement
- Human approves a gate policy recommendation without modification → agreement

What counts as a disagreement:

- Human edits an ActionContract stub before approving → disagreement
- Human overrides a verdict (e.g., manually approves a REJECTED ticket) → disagreement
- Human modifies a gate policy recommendation → disagreement

The gold label: Every disagreement produces a diff between the AI-generated version and the human-approved version. This diff is the highest-value signal in the entire loop — it shows exactly what the system got wrong in terms the system can learn from.

```
Gold Label:
  artifact_type:   "action_contract_stub"
  original:        { ... AI-generated ... }
  approved:        { ... human-edited ... }
  diff:            { field: "guard_predicates", added: ["balance_sufficient"] }
  timestamp:       2026-03-08T14:00:00Z
  reviewer_id:     "vin"
```

Gold labels are stored in `azul_data/gold_labels/` as append-only JSONL.

3.3 Drift detection (scheduled refinement)

In addition to threshold-based triggers, the analyzer runs a **weekly drift scan** (configurable cadence) that compares:

- Original AI-generated contract stubs vs. current approved contracts in `action_catalogs/`
- Original AI-proposed gate policies vs. current active policies in `config/gate_policies/`
- Original Dev briefing prompts vs. current versions

The drift scan catches slow degradation that never crosses an acute threshold but accumulates over time. Output: a drift report documenting what changed, by how much, and whether the changes represent improvement or regression.

```
Drift Report:
  scan_date:          2026-03-15
  contracts_compared: 42
  contracts_drifted:  7
  drift_direction:    "improvement" (6 tightened guards, 1 relaxed threshold)
  gold_labels_since:  12
  agreement_rate:     78% → 84% (improving)
  recommendation:     "Update stub generation prompt to include balance_sufficient guard pattern"
```

---

4 — Stage 4: IMPROVE (Four Levels)

4.1 Level 1 — Prompt improvement

Trigger: Reject rate for a ticket_type exceeds 30%, OR agreement rate below 70% for a specific artifact type.

Action: Update the AI Dev briefing documents and system prompts with:

- Examples of what passes (from high-scoring ReviewBundles)
- Examples of what fails (from REJECTED tickets with clear failure reasons)
- Gold label patterns (what humans consistently correct)

Input: ReviewBundles from recent tickets, gold labels from disagreements **Output:** Updated briefing documents in `docs/`, updated system prompts

Mechanism: The improvement recommender generates a prompt diff — a specific set of additions to the Dev's briefing or system prompt, with before/after examples. A human reviews and approves the update (this is itself subject to Lead Agreement Rate tracking).

4.2 Level 2 — Contract refinement

Trigger: Same oracle mismatch appears 3+ times, OR a specific ActionContract is involved in 3+ REJECTED tickets.

Action: Update the ActionContract's guard predicates, description, consistency profile, or compensation strategy based on the failure patterns.

Input: Oracle mismatch details from ReviewBundles, gold labels where humans added guard predicates **Output:** Updated YAML manifests in `action_catalogs/`

Mechanism: The recommender proposes a contract patch (specific field changes with rationale). The architect reviews and approves. ForgeAtlas reloads the catalog. The next cycle's verifications use the updated contract.

4.3 Level 3 — Policy tuning

Trigger: Gate policy override used 5+ times for the same reason, OR a domain's reject rate is anomalously high/low compared to others.

Action: Adjust gate policy thresholds in the domain's YAML configuration.

Input: Gate override history, cross-domain reject rate comparison **Output:** Updated YAML in `config/gate_policies/`

Mechanism: The recommender proposes a policy adjustment (specific threshold change with rationale and expected impact). The architect reviews and approves.

4.4 Level 4 — Training data (distillation)

Trigger: Verified training pairs accumulated past batch_size threshold (configurable, default 100).

Action: Feed verified pairs into a fine-tuning pipeline for local models.

Input: Training pairs from `azul_data/training_pairs/` with score > distillation_threshold (configurable, default 90) **Output:** Fine-tuned model checkpoint

Strategic outcome: Over time, you are building a proprietary model that knows your specific codebase (or your client's codebase) better than any frontier model. Each verification cycle adds to this asset. The model is trained exclusively on behaviorally verified examples — not internet scraped data, not synthetic benchmarks, but proven-correct executions against real code under real governance.

This is not just quality control. It is the construction of a codebase-specific intelligence that compounds with every cycle.

---

5 — Trigger Logic

The loop runs on two trigger mechanisms:

5.1 Threshold triggers (acute — respond fast)

```python
```

```
THRESHOLD_CONFIG = {
    # Reject rate triggers
    "reject_rate_threshold":        0.30,   # per ticket_type
    "reject_rate_window":           50,     # tickets

    # Agreement rate triggers
    "agreement_rate_low_threshold": 0.70,   # per artifact type
    "agreement_rate_window":        20,     # reviews

    # Oracle mismatch triggers
    "oracle_mismatch_repeat":       3,      # same mismatch appears N times

    # Policy override triggers
    "policy_override_repeat":       5,      # same override reason N times

    # Distillation triggers
    "distillation_batch_size":      100,    # training pairs accumulated
    "distillation_min_score":       90.0,   # minimum score for training data
}
```

After every ticket completes, the loop orchestrator checks whether any threshold has been crossed. If so, it triggers the corresponding LEARN + IMPROVE action.

5.2 Scheduled refinement (gradual — catch drift)

```
python

SCHEDULE_CONFIG = {
    "drift_scan_cadence":    "weekly",    # compare originals vs approved
    "full_analysis_cadence": "weekly",    # compute all metrics
    "report_cadence":        "weekly",    # emit improvement_report.json
    "distillation_cadence":  "monthly",   # batch fine-tuning run
}
```

The scheduled scan runs regardless of whether thresholds were crossed. It catches slow drift that accumulates below acute thresholds.

---

6 — Components to Build

6.1 Pattern Analyzer (`loop/pattern_analyzer.py`)

Reads Azul's ticket store, XP ledger, ReviewBundles, and gold labels. Computes all metrics from §3.1.

Produces a structured analysis report.

```python
def analyze(
    ticket_store: TicketStore,
    xp_ledger: XPLedger,
    gold_label_store: GoldLabelStore,
    window: int = 50,
) -> AnalysisReport:
    """
    Compute all metrics over the last N tickets.
    Returns structured report with findings and trigger flags.
    """
```

6.2 Improvement Recommender (`loop/recommender.py`)

Takes an AnalysisReport and generates specific, actionable improvement recommendations at all four levels.

```python
def recommend(
    report: AnalysisReport,
    current_contracts: ContractRegistry,
    current_policies: dict,
    current_prompts: dict,
) -> ImprovementPlan:
    """
    Generate specific recommendations: prompt diffs, contract patches,
    policy adjustments, distillation batch.
    Each recommendation includes rationale and expected impact.
    """
```

6.3 Gold Label Store (`loop/gold_labels.py`)

Captures and stores the diffs between AI-generated artifacts and human-approved versions.

```python
```

```
def record_label(
    artifact_type: str,
    original: dict,
    approved: dict,
    reviewer_id: str,
) -> GoldLabel:
    """
    Compute diff, store as gold label in append-only JSONL.
    """

def get_labels(
    artifact_type: str = None,
    since: datetime = None,
) -> list[GoldLabel]:
    """Query stored gold labels."""
```

6.4 Drift Scanner (`loop/drift_scanner.py`)

Compares original AI outputs against current approved versions on a scheduled cadence.

python

```
def scan_drift(
    catalog_dir: str,
    policy_dir: str,
    gold_label_store: GoldLabelStore,
    since: datetime,
) -> DriftReport:
    """
    Compare originals vs approved versions.
    Compute agreement rate trend.
    Identify systematic drift patterns.
    """
```

6.5 Loop Orchestrator (`loop/orchestrator.py`)

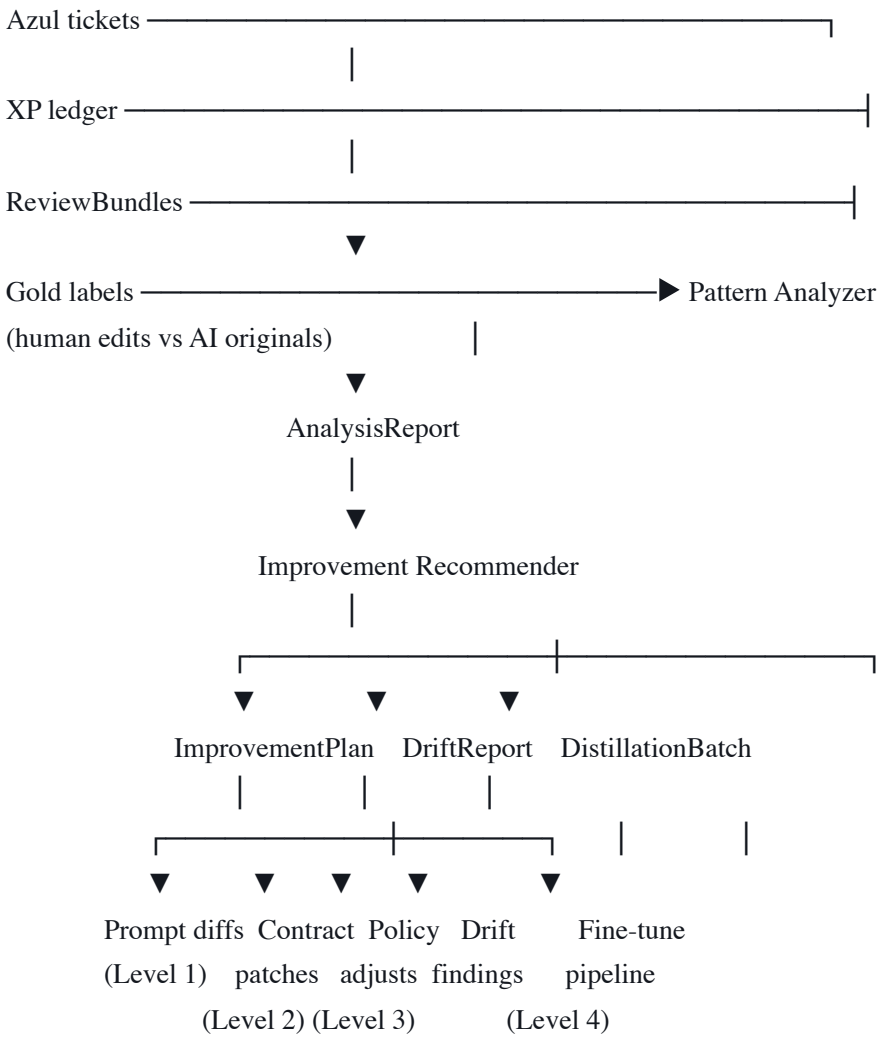
The daemon component that ties everything together. Runs threshold checks after every ticket and scheduled scans on cadence.

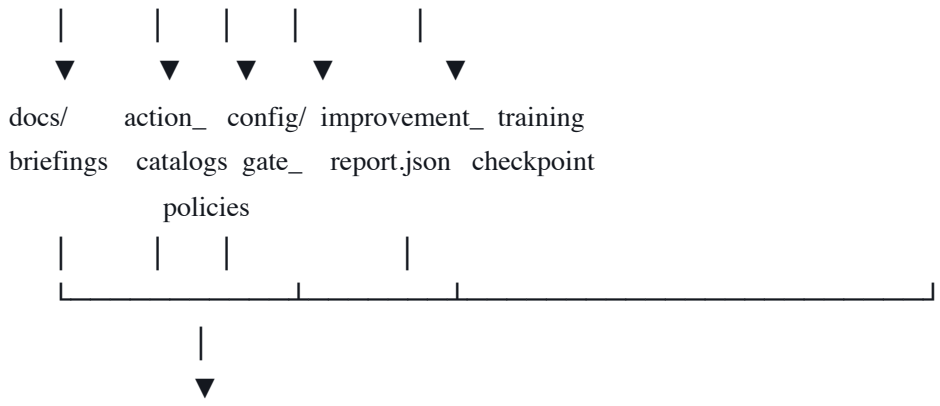
python

```
class LoopOrchestrator:
    """
    Monitors Azul ticket completions.
    After each ticket: check thresholds → trigger LEARN if crossed.
    On schedule: run drift scan → run full analysis → emit report.
    """

    def on_ticket_complete(self, ticket: AzulTicket):
        """Called by Azul after every verdict. Checks acute thresholds."""

    def run_scheduled_scan(self):
        """Periodic: drift scan + full analysis + improvement report."""

    def run_distillation_batch(self):
        """Monthly: collect verified pairs, prepare fine-tuning batch."""
```

---

7 — Data Flow





---

8 — Persistent Storage

```
azul_data/
├── gold_labels/
│   └── gold_labels.jsonl            ← append-only: human edits vs AI originals
├── improvement_reports/
│   └── report_2026-03-15.json       ← weekly analysis + recommendations
├── drift_reports/
│   └── drift_2026-03-15.json        ← weekly drift scan results
├── distillation_batches/
│   └── batch_2026-03-01/
│       ├── pairs.jsonl              ← verified training pairs for this batch
│       └── manifest.json            ← batch metadata (pair count, avg score, domains)
└── loop_state.json                  ← orchestrator state (last run times, counters)
```

---

9 — Success Metrics

The loop is working when these trends are observable over 30-day windows:

Metric	Target direction	Measurement
Lead Agreement Rate	Rising (60% → 95% over 30 days)	Gold labels: agreements / total reviews
Reject rate	Falling	REJECTED / total tickets per type
Average verification score	Rising	Mean total_score across tickets
XP velocity	Rising (same or fewer tickets, more XP per ticket)	XP per time window
Distillation yield	Rising	Training pairs emitted / total tickets
Operational failure rate	Falling	FAILED / total tickets
Gate override frequency	Falling	Manual overrides / total verdicts
Time to first verdict	Stable or falling	Median ticket duration

The top-line number is **Lead Agreement Rate**. If it's climbing, the loop is working. If it's flat, something in the LEARN → IMPROVE chain is broken.

---

10 — What the Loop Must NOT Do

Boundary	Reason
Auto-apply prompt updates without human review	Prompts shape agent behavior. Unsupervised prompt modification is an uncontrolled feedback loop.
Auto-modify gate policies without human review	Gate policies are governance. Automatic loosening could silently degrade safety.
Auto-merge contract changes	ActionContracts define behavioral expectations. Human domain knowledge is required.
Train on REJECTED tickets without explicit labeling	Rejected tickets are negative examples. They should be labeled as such, not mixed with positive training data.
Override Azul verdicts based on loop recommendations	The loop improves the system for the next cycle. It does not retroactively change past verdicts.
Run distillation fine-tuning without human approval	Model updates are irreversible in practice. The architect approves each fine-tuning batch.

Every IMPROVE action produces a recommendation. A human reviews and approves it. The loop accelerates human judgment — it does not replace it.

---

11 — Configuration

11.1 Environment variables

Variable	Default	Purpose
`FORGE_LOOP_ENABLED`	`true`	Master switch for the recursive loop
`FORGE_LOOP_THRESHOLD_WINDOW`	`50`	Tickets analyzed per threshold check
`FORGE_LOOP_REJECT_THRESHOLD`	`0.30`	Reject rate that triggers Level 1
`FORGE_LOOP_AGREEMENT_THRESHOLD`	`0.70`	Agreement rate below which triggers Level 1
`FORGE_LOOP_MISMATCH_REPEAT`	`3`	Oracle mismatch repeat count for Level 2
`FORGE_LOOP_OVERRIDE_REPEAT`	`5`	Policy override repeat count for Level 3
`FORGE_LOOP_DISTILL_BATCH_SIZE`	`100`	Training pairs per distillation batch
`FORGE_LOOP_DISTILL_MIN_SCORE`	`90.0`	Minimum score for distillation data
`FORGE_LOOP_DRIFT_CADENCE`	`weekly`	Drift scan frequency
`FORGE_LOOP_REPORT_CADENCE`	`weekly`	Full analysis report frequency
`FORGE_LOOP_DISTILL_CADENCE`	`monthly`	Fine-tuning batch frequency

---

12 — Integration with Existing Components

Component	Loop interaction
Azul	Loop orchestrator hooks into `on_ticket_complete`. Reads ticket store, XP ledger, training pairs.
ForgeAtlas	Level 2 improvements update YAML manifests. ForgeAtlas reloads on catalog change.
CONCORD	Level 3 improvements update gate policy YAMLs consumed by FWResultGate.

Component	Loop interaction
ForgeScaffold	Drift scanner compares original scan outputs against current contract state.
ForgeWorks	ReviewBundles are the primary evidence source for the pattern analyzer.
ForgeGate	Per-phase deny rates feed into the pattern analyzer.
ForgeHarbor	Operational failure analysis (FAILED tickets) identifies environment provisioning friction.
DAWN	Ledger events provide timing data for time-to-verdict metrics.

End of core specification. Four stages (Execute → Verify → Learn → Improve). Five new components (Pattern Analyzer, Improvement Recommender, Gold Label Store, Drift Scanner, Loop Orchestrator). Lead Agreement Rate is the top-line metric. Every improvement requires human approval. The loop builds a proprietary intelligence asset that compounds with every cycle.